

Lecture 4: Support vector machine and k-nearest neighbor algorithms

**01211373 : Machine Learning and
Programming for Industry**

Dr. Varodom Toochinda

Dept. of Mechanical Engineering,
Kasetsart University



Topics

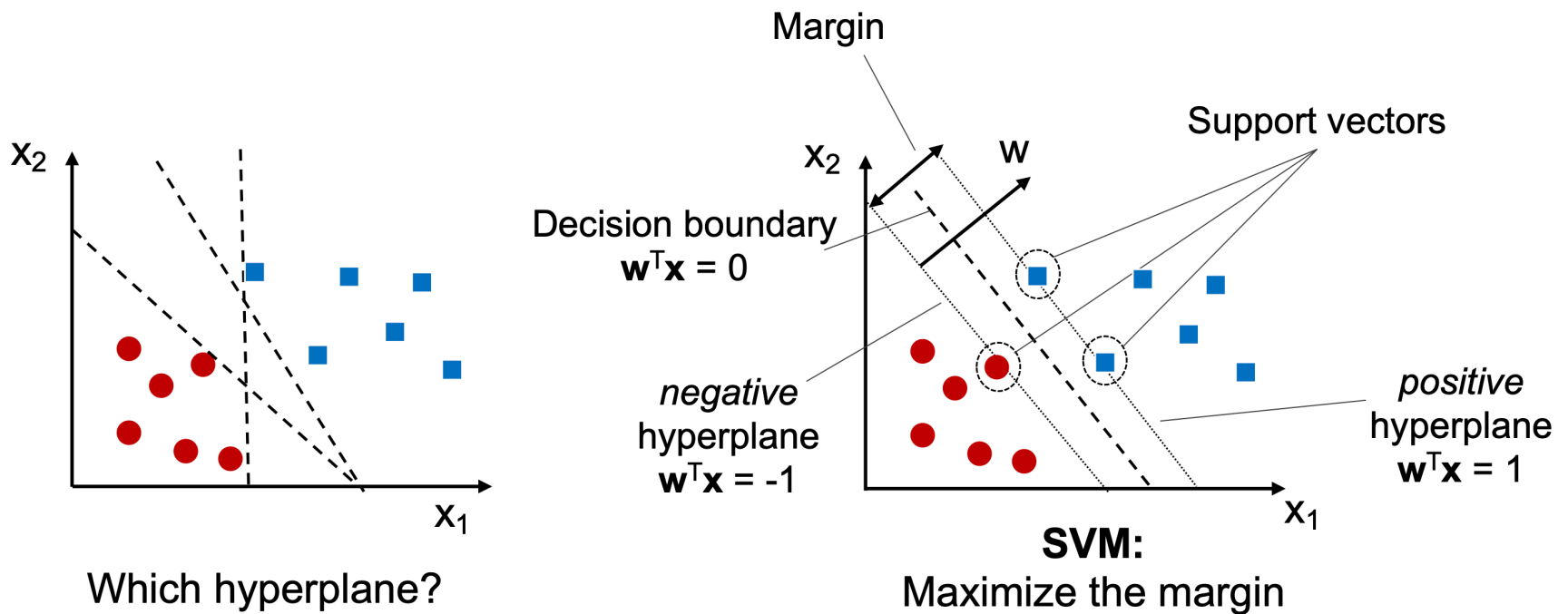
- ❑ Support Vector Machine (SVM)
- ❑ K-nearest neighbor

Support Vector Machines

From Wikipedia https://en.wikipedia.org/wiki/Support_vector_machine

"In machine learning, support vector machines (SVMs, also support vector networks[1]) are supervised max-margin models with associated learning algorithms that analyze data for classification and regression analysis. In addition to performing linear classification, SVMs can efficiently perform non-linear classification using the kernel trick, representing the data only through a set of pairwise similarity comparisons between the original data points using a kernel function, which transforms them into coordinates in a higher-dimensional feature space. Thus, SVMs use the kernel trick to implicitly map their inputs into high-dimensional feature spaces, where linear classification can be performed."

Support Vectors



From Fig 3.10 of [1]

Functional margins (of one example)

For one training example, **the functional margin** of hyperplane defined by (w, b) w.r.t $(x^{(i)}, y^{(i)})$ is

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x^{(i)} + b)$$

When compared to (4.3), with change of notation to $y \in \{-1, +1\}$, what we hope for is

$$\text{if } y^{(i)} = 1 \rightarrow w^T x^{(i)} + b \gg 0$$

$$\text{if } y^{(i)} = -1 \rightarrow w^T x^{(i)} + b \ll 0$$

We can combine the two statements in (4.7), together with (4.6). In order to have a confident prediction,

$$\hat{\gamma}^{(i)} \gg 0$$

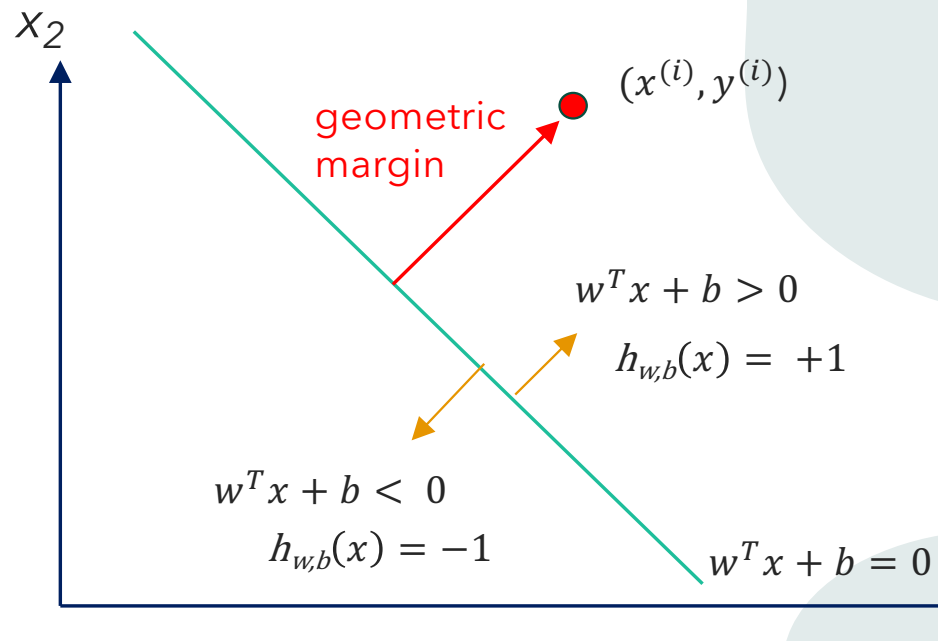
In fact, if $\hat{\gamma}^{(i)} > 0$, that means $h(x^i) = y^{(i)}$, the prediction is correct.

Functional margins (of the dataset)

Functional margin w.r.t the whole training set is defined as the worst-case margin

$$\hat{\gamma} = \min_i \hat{\gamma}^{(i)}, \quad i = 1, \dots, n$$

Geometric margin



Geometric margin (of one example)

Therefore, the geometric margin of hyperplane defined by (w, b) w.r.t one example $(x^{(i)}, y^{(i)})$ is described by

$$\gamma^{(i)} = \frac{(w^T x^{(i)} + b)}{\|w\|}$$

This is how the Euclidean distance from the decision line to example i is computed. See [2] for the proof.

More generally, the term $y^{(i)}$ is added to (4.11) to become

$$\gamma^{(i)} = \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|}$$

so that the definition (4.12) applies to both positive and negative examples. So the relationship between the geometric and functional margins is

$$\gamma^{(i)} = \frac{\hat{\gamma}^{(i)}}{\|w\|}$$

Geometric margin of the dataset

$$\gamma = \min_i \gamma^{(i)}, \quad i = 1, \dots, n$$

Formulation of optimal margin classifier

$$\begin{array}{ll} \max_{\gamma, w, b} & \gamma \\ \text{s.t.} & \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|} \geq \gamma \quad i = 1, \dots, n \end{array}$$

not a convex optimization problem

Equilalent optimization problem (convex)

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad i = 1, \dots, n \end{aligned}$$

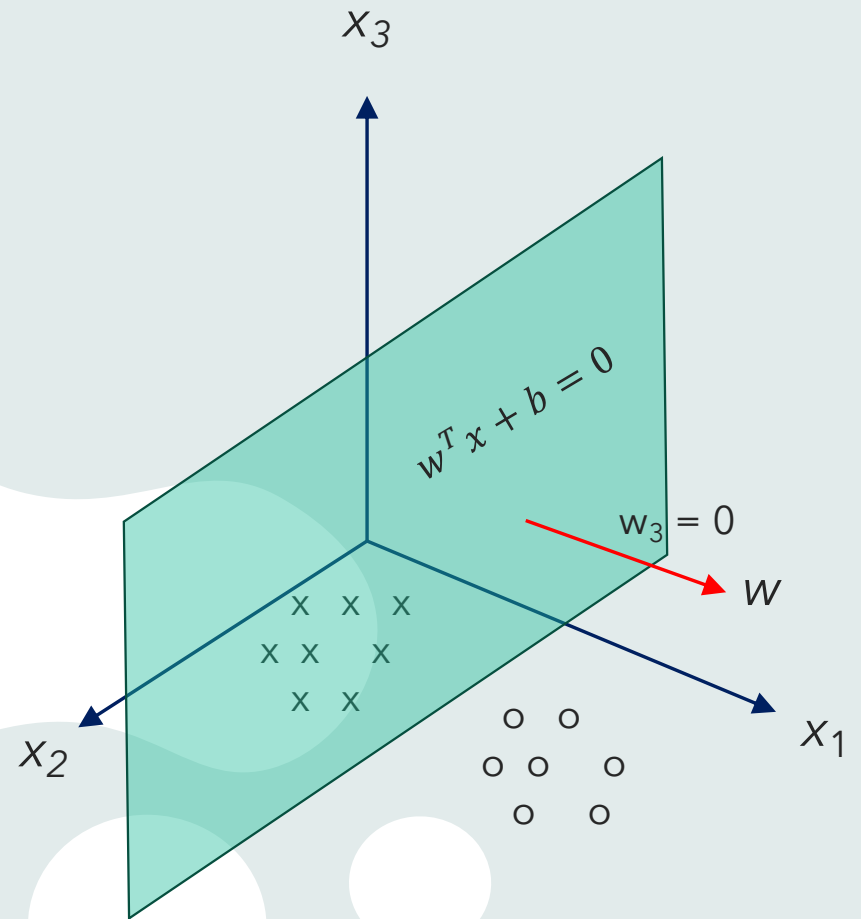
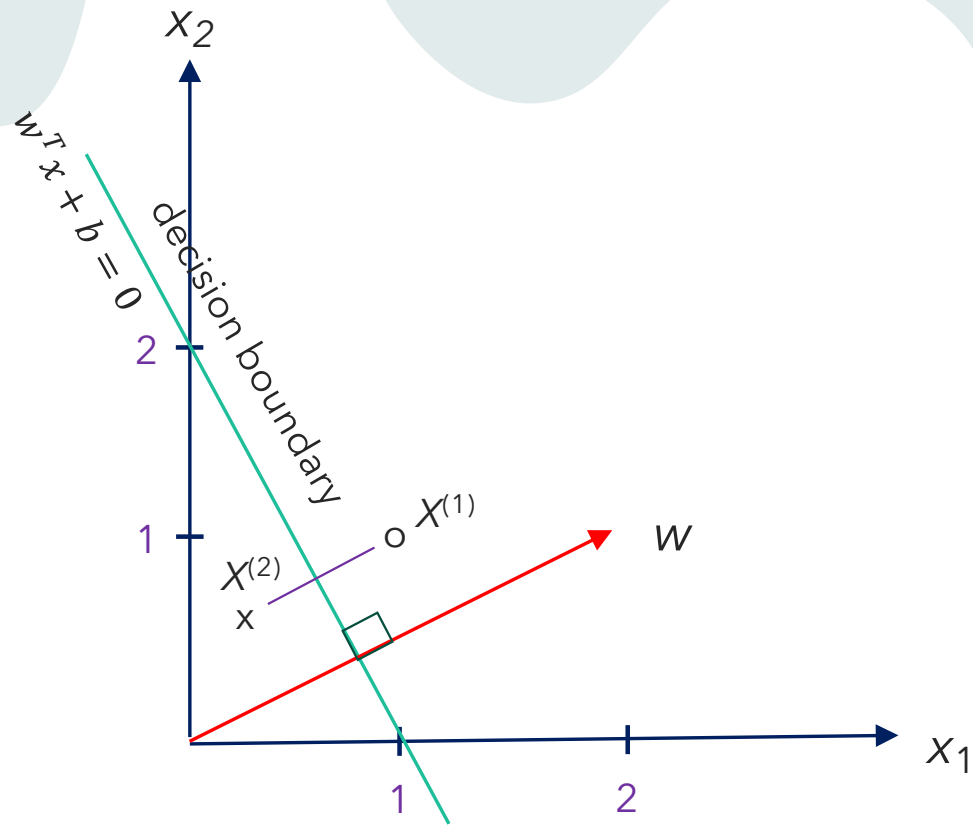
details in notebook and [2]

The Representer theorem

$$w = \sum_{i=1}^M y^{(i)} \alpha_i x^{(i)}$$

w can be expressed as a linear combination of the training examples (and we add $y = \{+1, -1\}$ just to make the math easier.

Intuition # 2



Simplified “dual” optimization problem

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)} x^{(j)} \rangle$$

$$s.t. \alpha_i \geq 0, \quad i = 1, \dots, n$$

$$\sum_i \alpha_i y^{(i)} = 0$$

To make a prediction

1. Solve for α_i and b
2. Compute $h_{w,b}(x)$ for new test example

$$\begin{aligned}h_{w,b}(x) &= g(w^T x + b) \\&= g \left(\left(\sum_i \alpha_i y^{(i)} x^{(i)} \right)^T x + b \right) \\&= g \left(\sum_i \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b \right)\end{aligned}$$

Kernel tricks

1. Write your whole algorithm in terms of $\langle x^{(i)}, x^{(j)} \rangle$. Or in general, we write it as $\langle x, z \rangle$ for inner product of two vectors x, z .
2. Let there be some mapping $x \rightarrow \phi(x)$ from low dimensional feature space x to higher dimensional $\phi(x)$. See some examples from "[Lecture 7 of CS229 video](#)".
3. Find a way to compute **the kernel function** $K(x, z) = \phi(x)^T \phi(z)$. We'll see that there are clever tricks to compute the inner product between x and z even when $\phi(x), \phi(z)$ are incredibly high dimensional.
4. Replace $\langle x, z \rangle$ in algorithm with $K(x, z)$. In essence, we're running the whole learning algorithm on this high dimensional set of features. It can be very computationally expensive working with high dimensional feature vectors $\phi(x)$. But instead we don't have to compute it explicitly. We can always just compute the kernel $K(x, z)$

see notebook and [2]

SVM structure

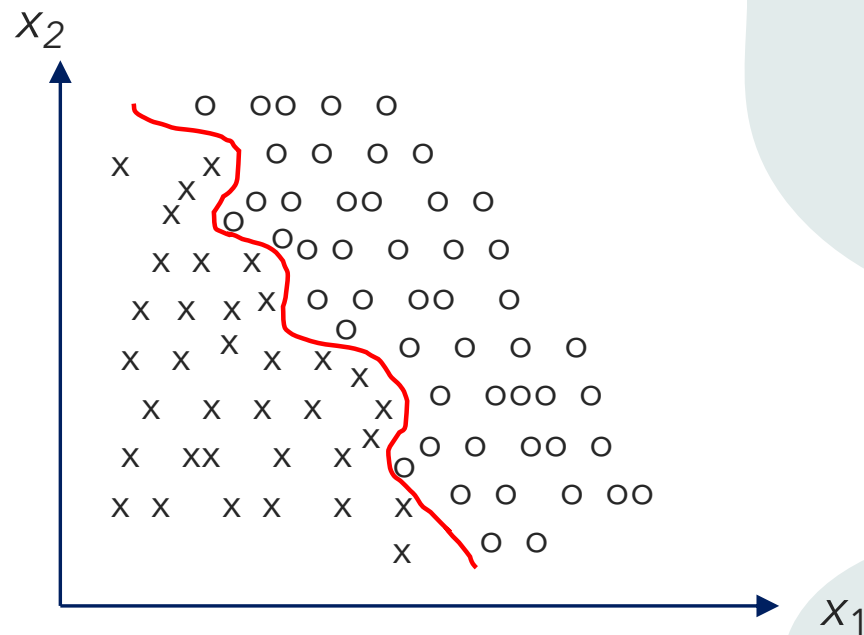
optimal
margin
classifier

+

kernel
tricks

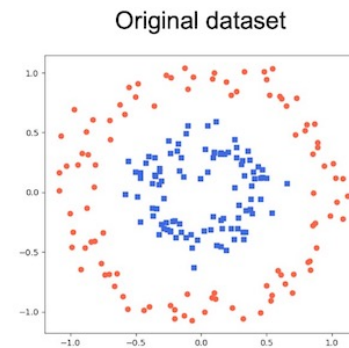
= SVM

SVM can learn nonlinear decision boundary



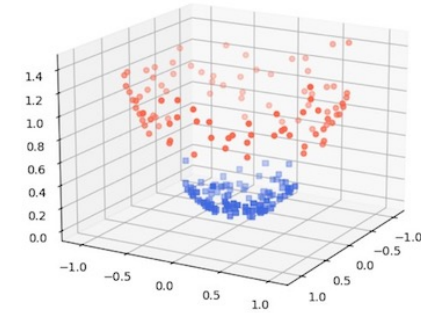
Use kernel tricks for nonlinear classification

<https://youtu.be/OdINM96sHio?si=vE8uRKGhL4ljFlrz>

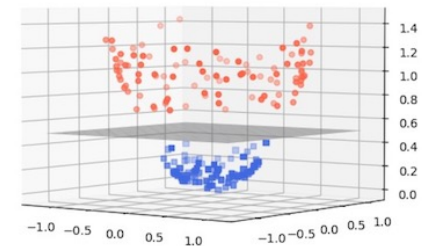


ϕ

Projection into higher-dimensional space

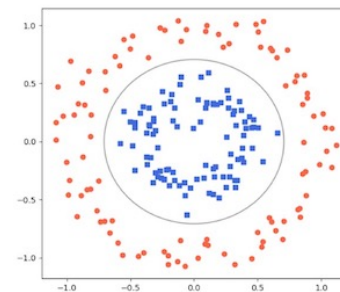


↓



Learn decision boundary (here: hyperplane)

ϕ^{-1}



Decision boundary projected in original feature space

Widely-used kernels

Linear kernel

$$K(x, z) = x^T z$$
$$\phi(x) = x$$

Polynomial kernel

$$K(x, z) = (x^T z)^d$$

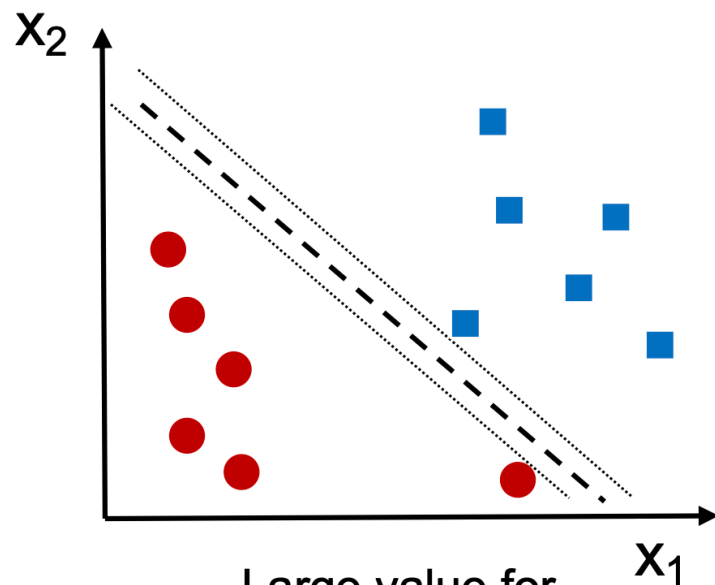
Gaussian, or Radial
Basis Function (RBF)

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

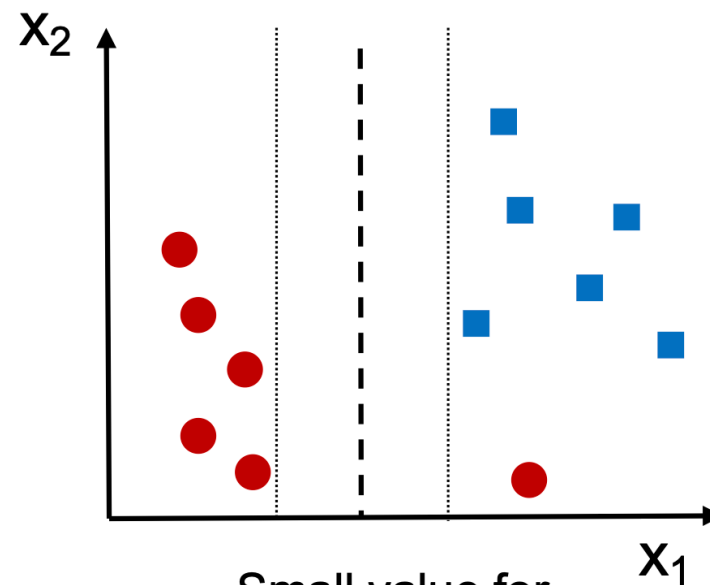
Soft-margin SVM

$$\begin{aligned} \min_{w,b,\xi_i} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, n, \\ & \xi_i \geq 0 \end{aligned}$$

Impact of C on regularization

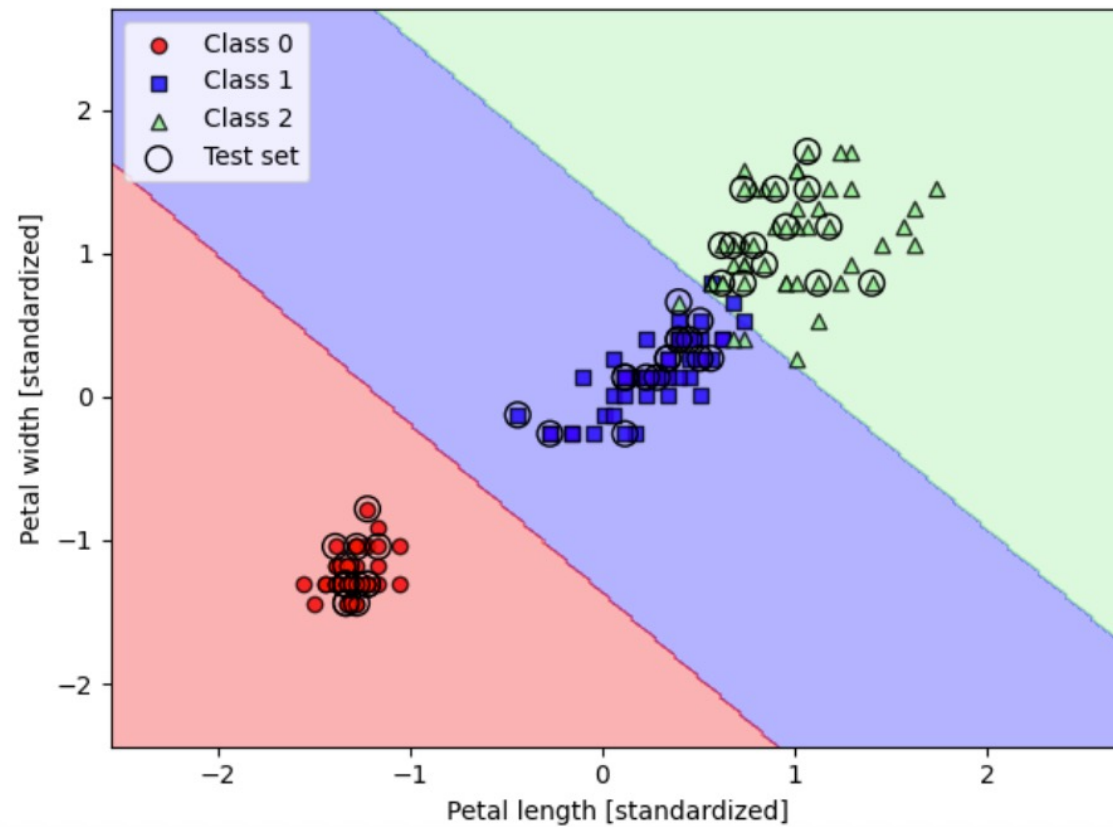


Large value for
parameter C



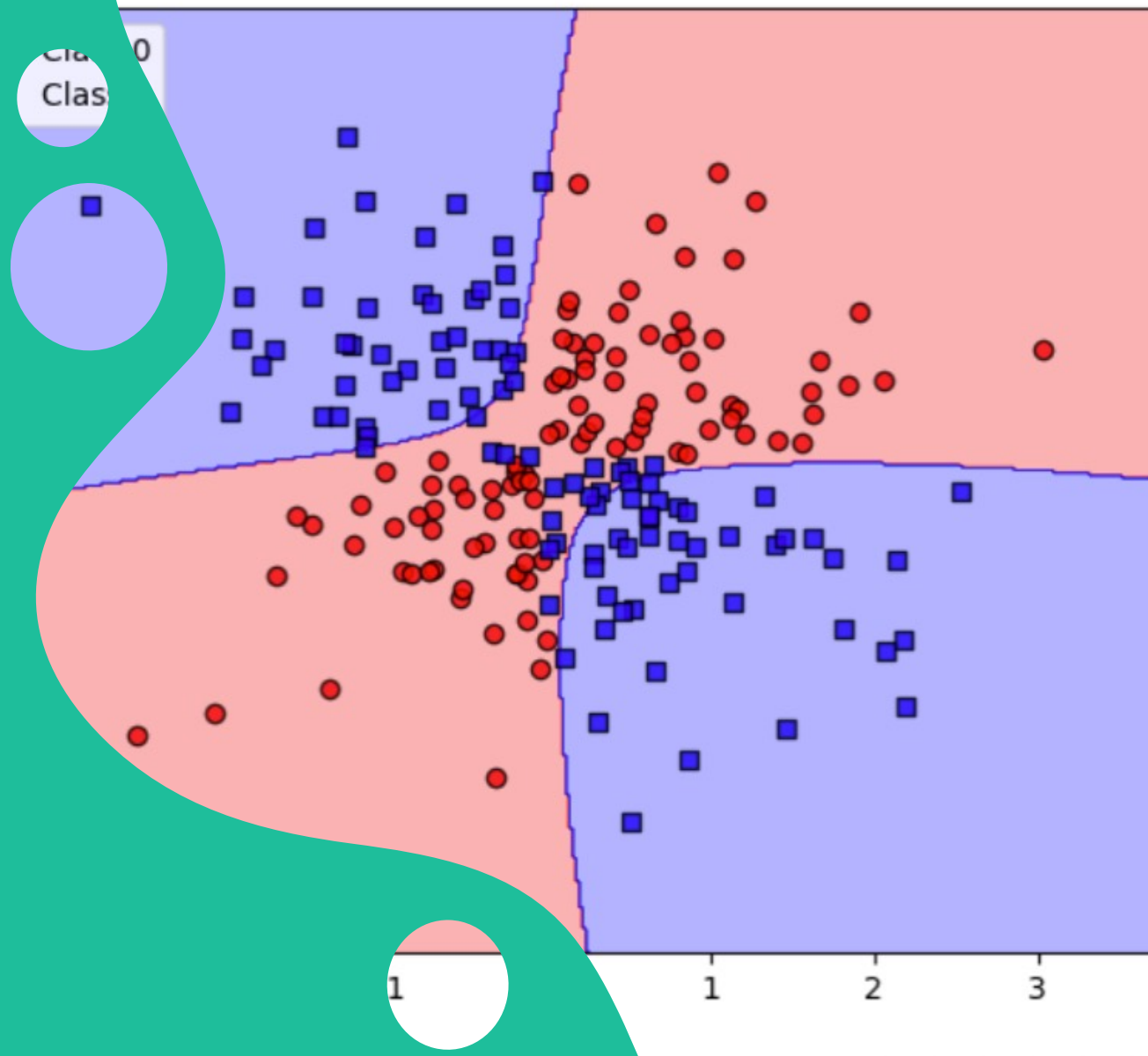
Small value for
parameter C

SVM using scikit-learn



classify the iris dataset

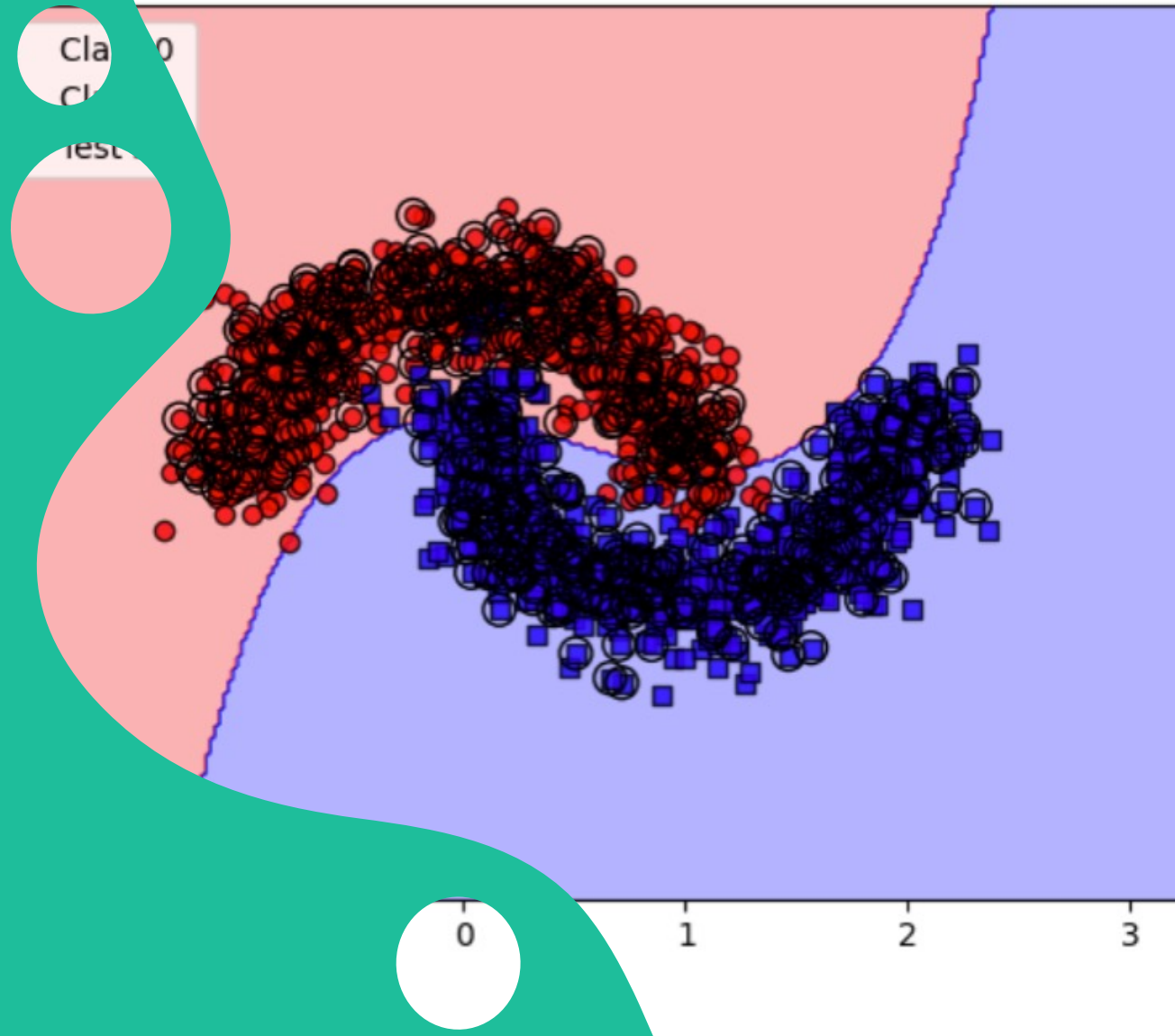
Use kernel
tricks to
classify
nonlinear
boundary



classify the moon dataset in scikit-learn

HW #3

Classification of moon dataset with RBF kernel





k-nearest neighbor algorithm

From wikipedia :

https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

"In statistics, the k-nearest neighbors algorithm (k-NN) is a non-parametric supervised learning method. It was first developed by Evelyn Fix and Joseph Hodges in 1951,[5] and later expanded by Thomas Cover.[6] Most often, it is used for classification, as a k-NN classifier, the output of which is a class membership. An object is classified by a plurality vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors (k is a positive integer, typically small). If $k = 1$, then the object is simply assigned to the class of that single nearest neighbor."

5. E. Fix and J.L. Hodges. Discriminatory Analysis. Nonparametric Discrimination: Consistency Properties (Report) . USAF School of Aviation Medicine, Randolph Field, Texas. 1951.

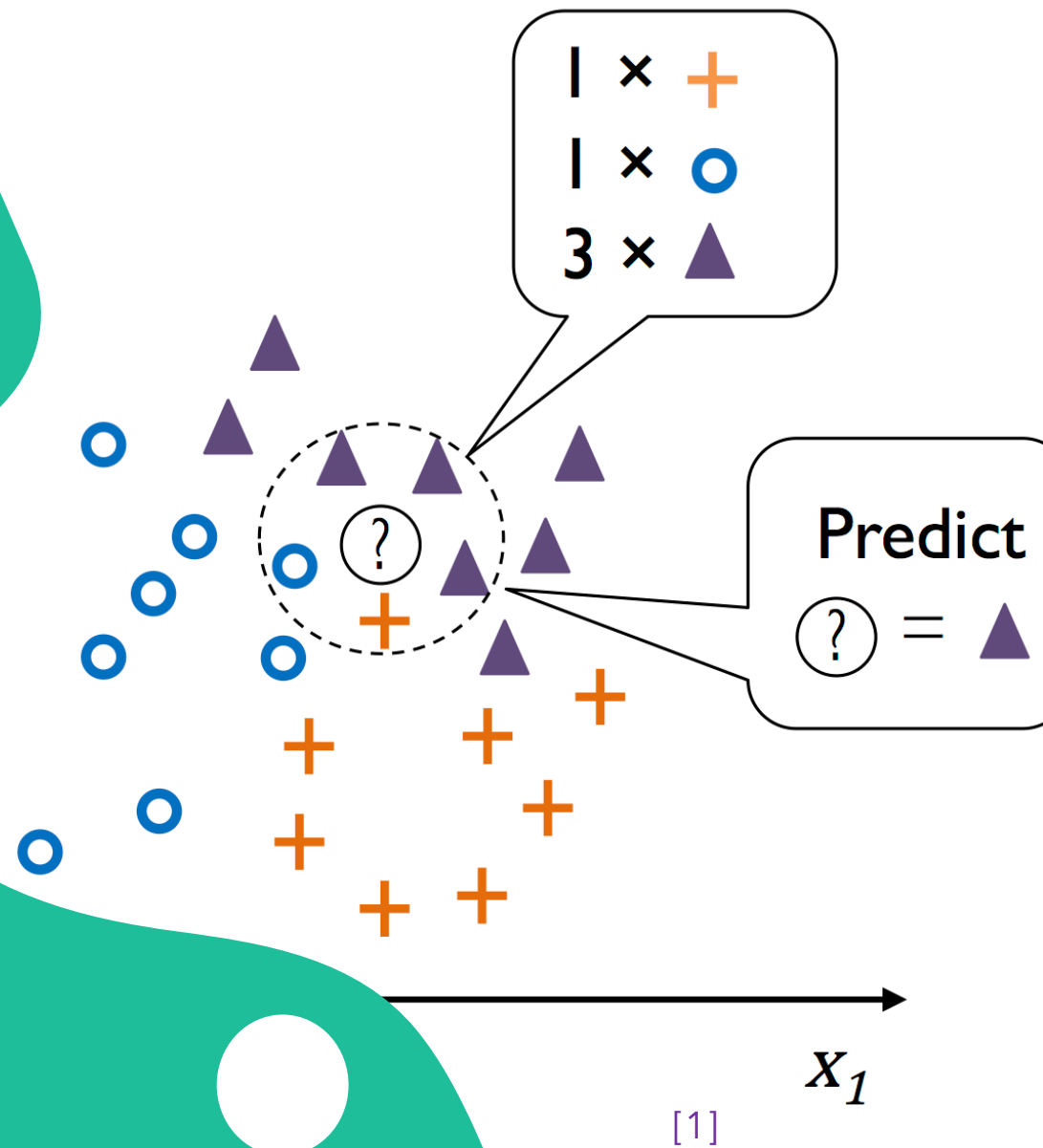
6. T.M. Cover and P.E. Hart. "Nearest neighbor pattern classification". IEEE Transactions on Information Theory. 13 (1): 21-27. 1967.

KNN algorithm

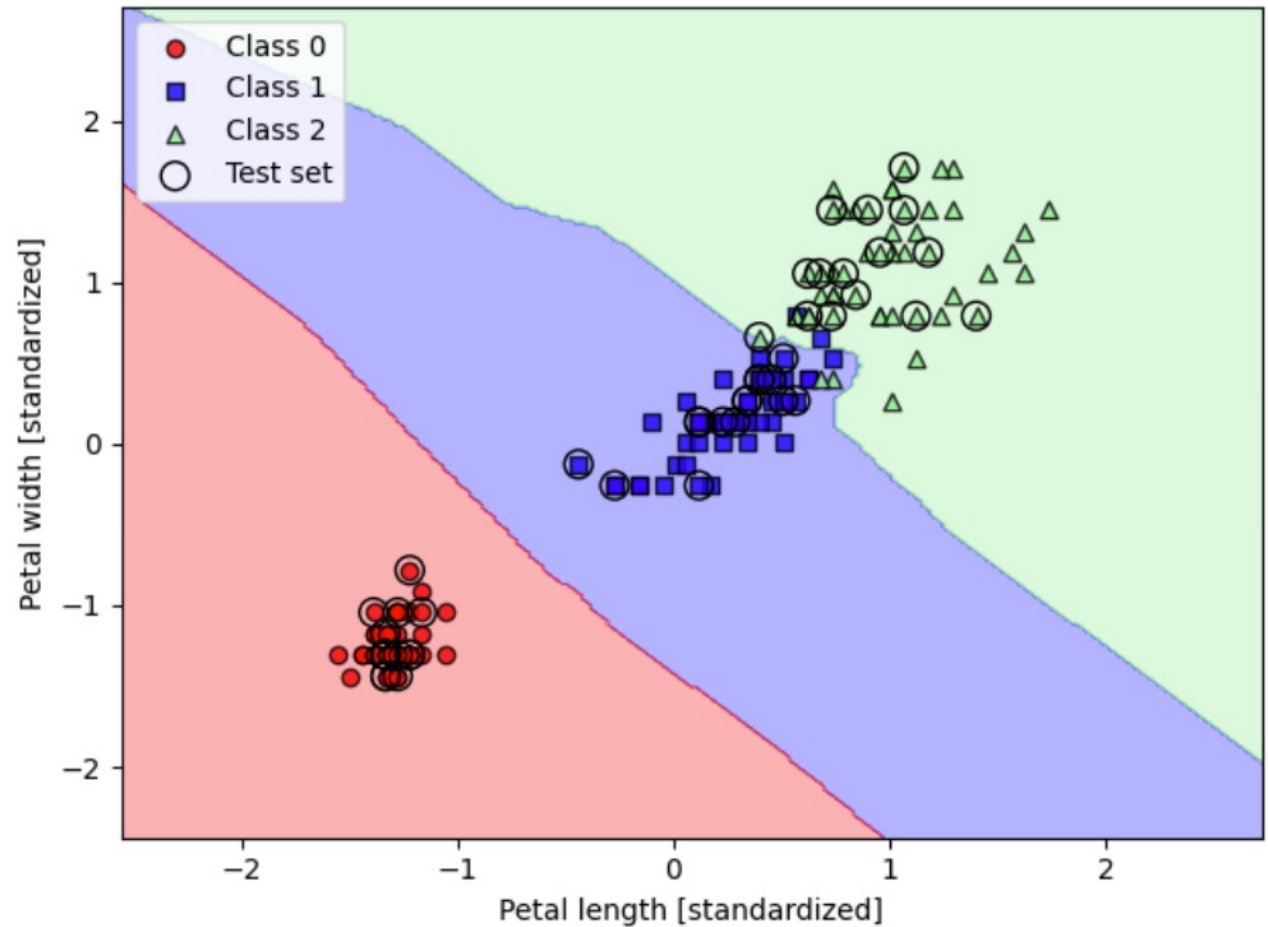
The KNN algorithm itself is fairly straightforward and can be summarized by the following steps:

1. Choose the number of k and a distance metric
2. Find the k -nearest neighbors of the data record that we want to classify
3. Assign the class label by majority vote

How k-nearest neighbor works



Using scikit-learn on Iris dataset



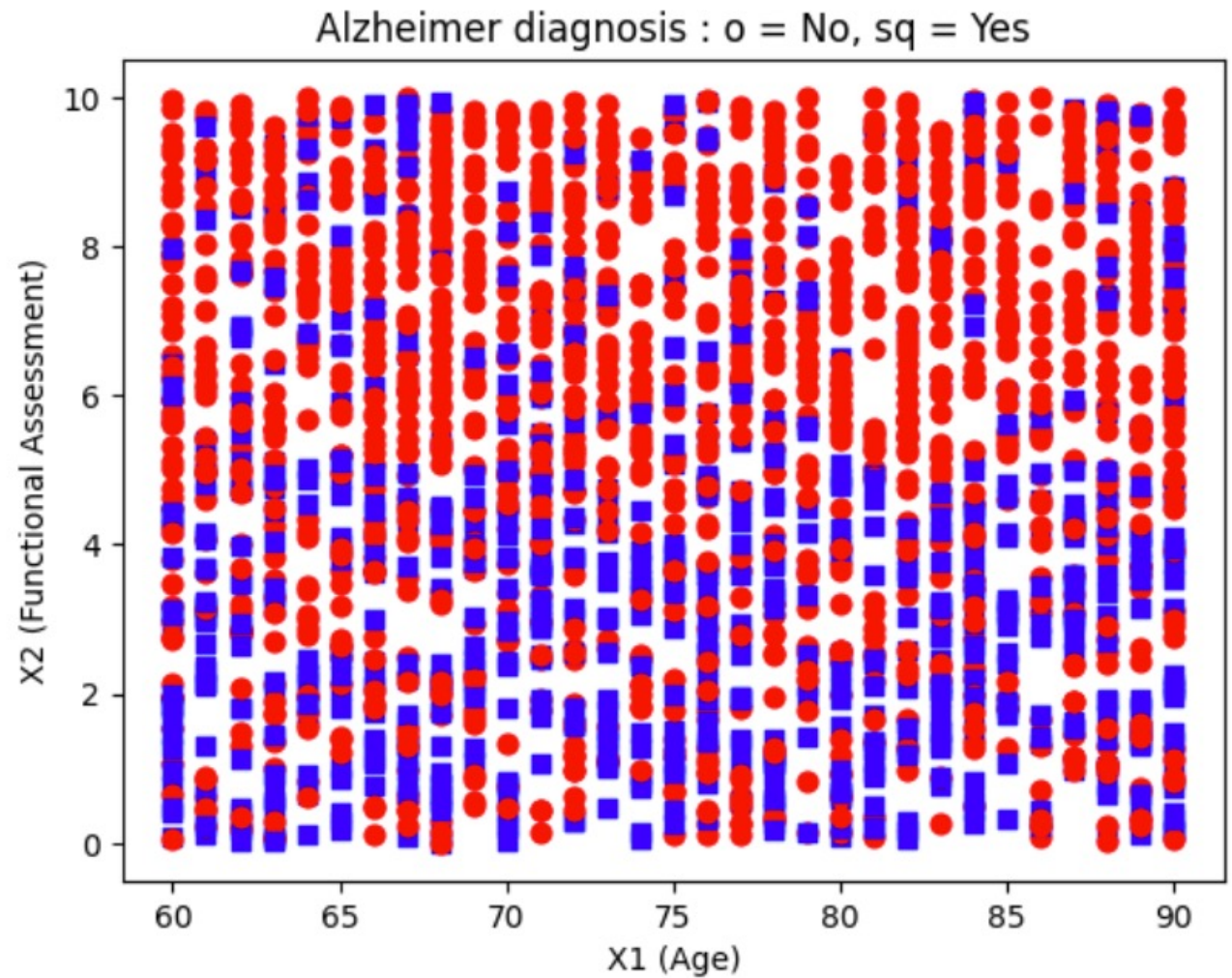
Experiments on Alzheimer's Disease Dataset

<https://www.kaggle.com/datasets/rabieelkharoua/alzheimers-disease-dataset>

In order to plot in 2D, choose "Age" and "FunctionalAssessment" as two features X1 and X2.

y is the "Diagnosis" column.

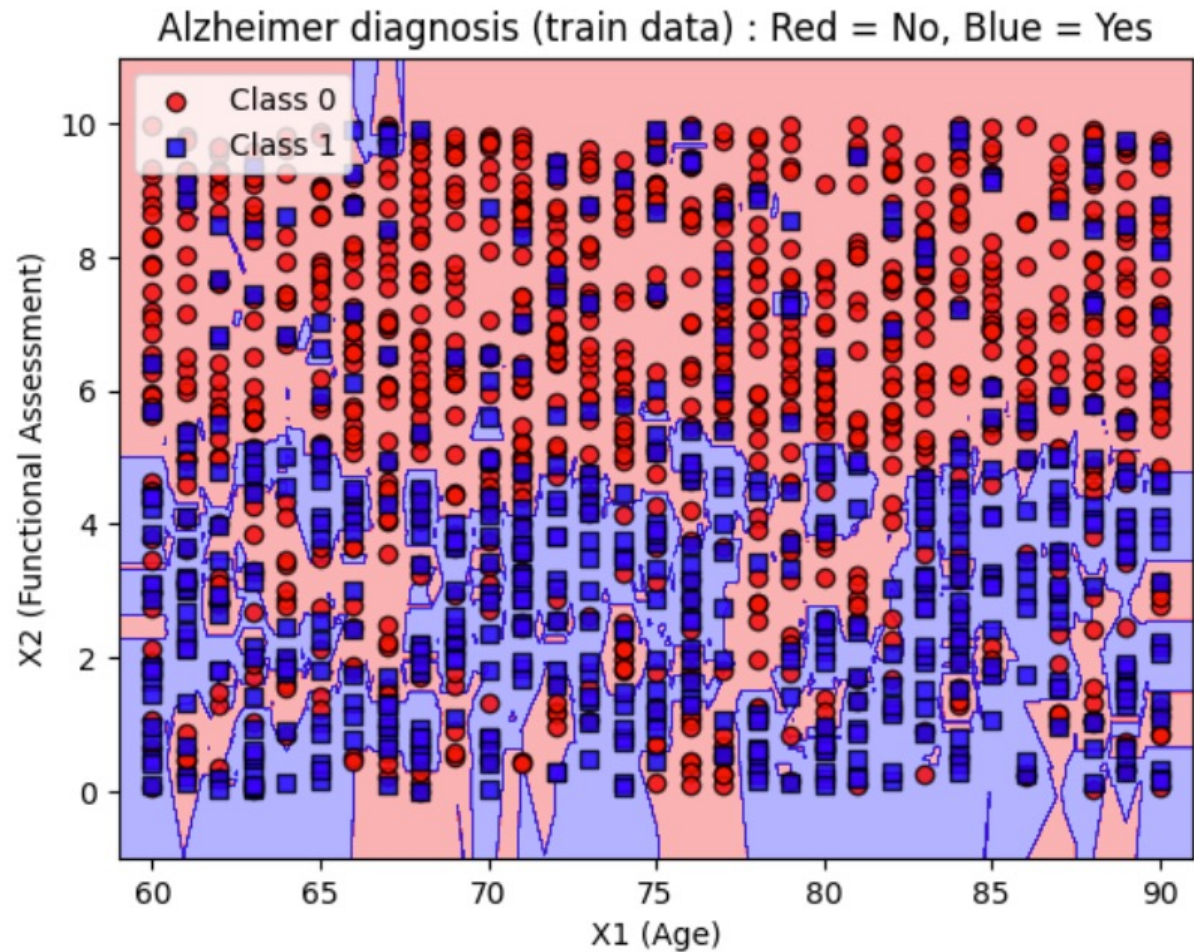
Initial data visualization



Classification result v.s train data

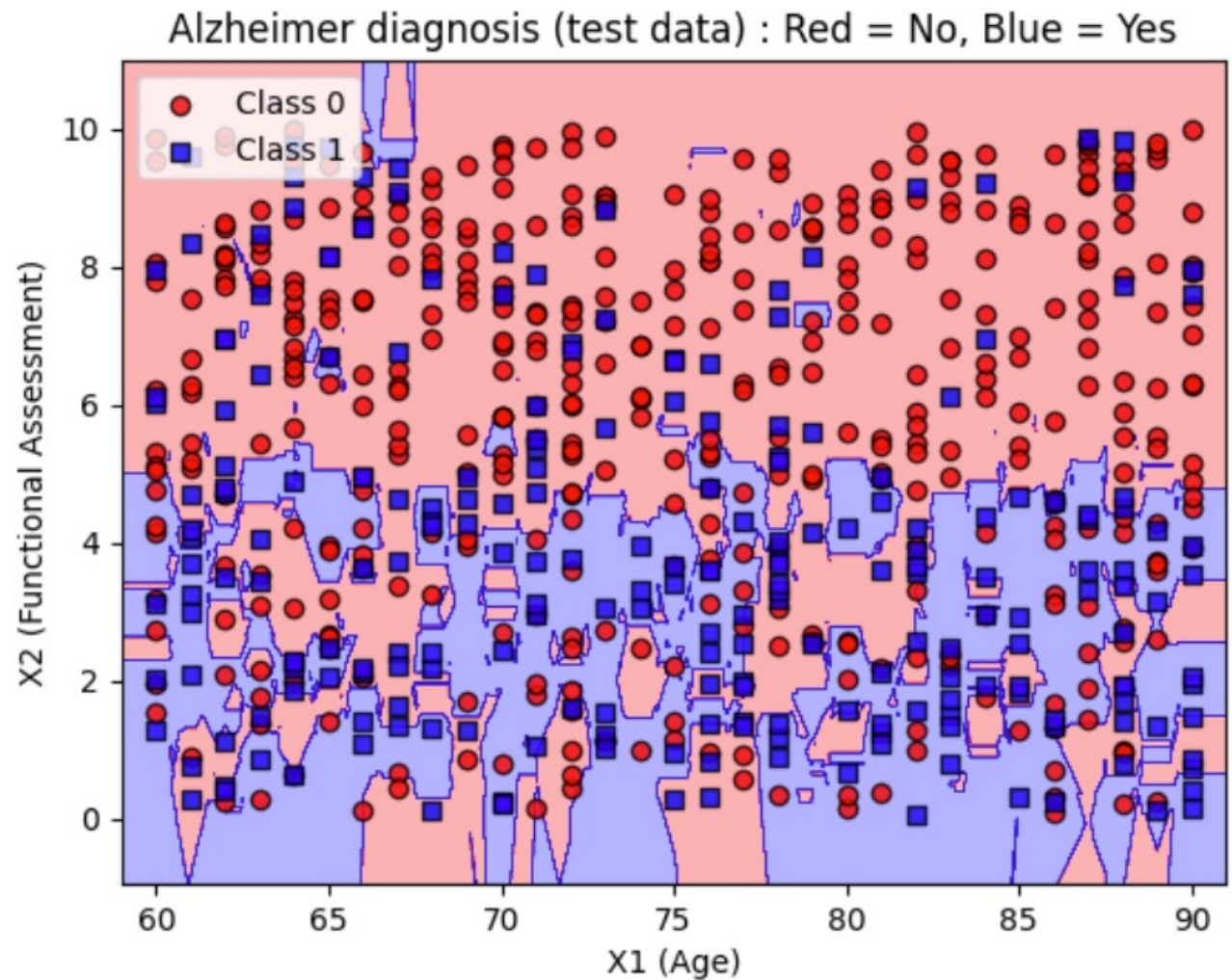
$k = 5$,
metric = 'minkowski'

train accuracy = 0.78



Classification result v.s test data

test accuracy = 65



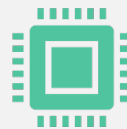
References



1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn. Packt Publishing. 2022.
<https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>



2. A. Ng. CS229 Lecture Notes. Stanford University. 2023.
https://cs229.stanford.edu/main_notes.pdf



3. T. Broderick. Introduction to Machine Learning course (6.036). Massachusetts Institute of Technology. 2020.
<https://tamarabroderick.com/ml.html>